

Chapter 1. MLOps Principles and Components

Machine learning operations (MLOps) is a set of practices that help to bring ML models to production in a scalable and efficient way. It can be seen as an extension of the development and operations (DevOps) philosophy that started around the end of 2000 as a movement against the established, traditional software development model, in which code developers and developers who deploy and support the code are separated functionally and organizationally. Siloed teams—with their own planning and key performance indicators (KPIs), inability to communicate code to the operations team, and misaligned development and production environments—contributed to long release cycles and created endless frustration.

MLOps addresses the same core challenge as DevOps did: building systems that can be reliably deployed, operated, and improved over time in production. The difference is that AI systems introduce additional parties—such as data scientists, ML engineers, and other specialized roles—as well as new types of challenges related to evaluation, testing, and reproducibility.

Unlike traditional software systems, ML applications are built not only from code but also from data. Their behavior is inherently probabilistic, which makes it significantly harder to define expected outcomes, write meaningful tests, or validate system performance before deployment.

These characteristics make the development lifecycle of ML systems fundamentally different from that of traditional software. This means that there needs to be closer collaboration between data practitioners and platform or DevOps teams to ensure that models can be deployed, monitored, and maintained in a reliable and repeatable way.

More broadly, these challenges extend to AI systems as a whole. Since ML is a subset of AI and forms the foundation of most modern AI systems, the same principles apply, even as systems become more complex.

In this chapter, I will walk you through the evolution of MLOps, explain where and why it diverges from traditional DevOps practices, and outline the key components required to operationalize ML systems. By the end of the chapter, you will understand why these components exist, which MLOps principles they support, and how to implement them within the Databricks platform.

The Rise of MLOps

The term *MLOps* began gaining widespread popularity around 2020. This was roughly five years after many large organizations started heavily experimenting with and investing in data science, only to realize that their return on investment (ROI) was often lower than expected. While building ML models proved relatively straightforward, integrating them into production systems in a reliable and scalable way turned out to be far more difficult.

In practice, bringing ML models into production requires far more than the model itself. ML code produced by data scientists represents only a small fraction of a real-world ML system. This is illustrated in [Figure 1-1](#), which is an adapted diagram from the well-known Google paper “Hidden Technical Debt in Machine Learning Systems.”¹ It shows that the majority of the system consists of surrounding infrastructure such as data pipe-

lines, validation logic, monitoring, serving layers, and integration components.

To make the outputs of ML models valuable to a business, models must be integrated into existing software applications or entirely new applications must be built around them. This is a responsibility that has traditionally fallen within the domain of DevOps teams.

Historically, data analytics and later data science were closely aligned with business functions. As a result, many data scientists had limited exposure to software engineering best practices and the broader IT landscape. Deploying scalable ML applications often required formal handovers to DevOps teams, and that created organizational bottlenecks. In many ways, this mirrored the disconnect that once existed between development and IT operations teams prior to the DevOps movement.

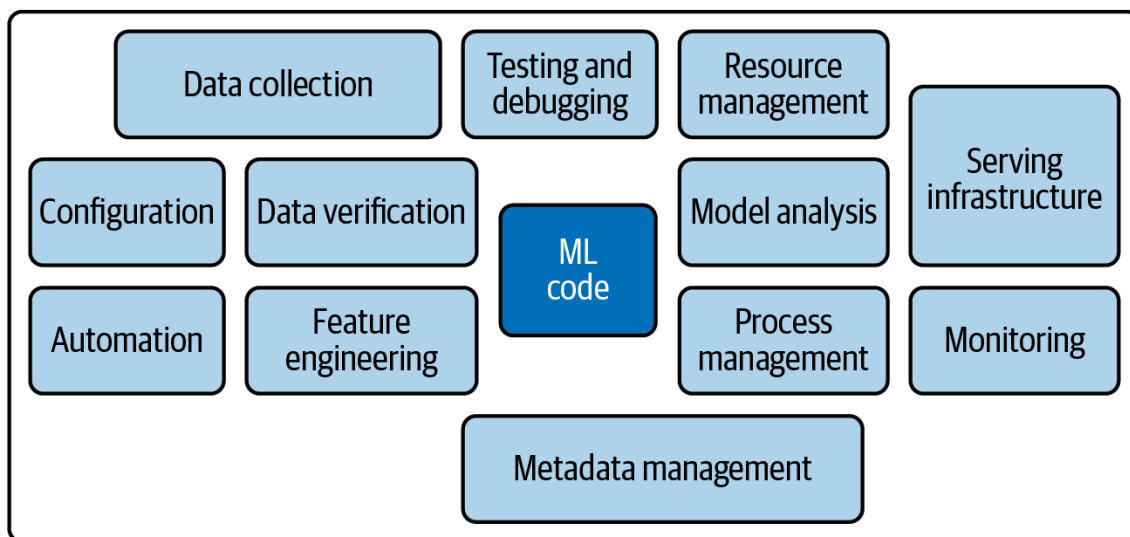


Figure 1-1. Diagram showing ML code represents only a small fraction of a real-world ML system

A shift was therefore needed. In recent years, many organizations have begun embracing MLOps practices by moving away from centralized data science teams and toward more product-centric, cross-functional teams. This shift has given rise to the role of the *machine learning engineer*, which is embedded within these teams to enable end-to-end ownership of ML-powered products.

At the same time, platform teams have started adopting MLOps principles by building and supporting internal tooling that enables ML product teams to reduce time to production. As a result, a broad ecosystem of tools and platforms has emerged to support the operationalization of ML systems. Similarly, leading tech companies have built end-to-end platforms

for their ML operations, some which have become iconic in their influence in the field. Examples include Uber’s Michelangelo,² LinkedIn’s ProML,³ and Netflix’s ML Platform.⁴

Today, organizations are in a similar phase, this time with AI applications. Many teams are actively experimenting with LLMs and other AI technologies, and it is likely that a stronger operational focus will follow soon, probably faster than it did in the case of MLOps.

Because many organizations have already undergone the structural shift toward product-oriented teams and platform enablement, the adoption of LLMOps practices is expected to accelerate. In addition, the ecosystem of tools and platforms available today is significantly more mature than it was during the early wave of ML adoption, so it’s easier for organizations to move from experimentation to reliable, production-grade AI systems.

MLOps Principles

We’ve established how MLOps became a discipline in its own right, so now, let’s look at its principles. These principles matter because companies rely on ML models as part of their business operations, and real negative business impacts follow when model predictions are unavailable or of poor quality.

There are many examples out there of how things can go wrong. Zillow, a real estate company, lost \$500 million in 2021 due to houses that were bought at too high a price because of inaccurate ML predictions.⁵

The goal of MLOps is to minimize the risks of things going wrong. This brings us to the *four main MLOps principles*: reliability, traceability, reproducibility, and observability. Each principle provides the answer to its own core question (see [Table 1-1](#)).

Table 1-1. The four main MLOps principles and the core questions they answer

Principle	Core question
Traceability	What happened?
Reproducibility	Can we recreate it?
Reliability	Can we depend on it?
Observability	Can we understand and diagnose it?

These are developer-centric principles and focused on what engineering teams can control and implement. Broader ethical principles, such as fairness, privacy, and accountability, are equally important but fall outside the scope of this book. The Linux Foundation's Trusted AI Reproducibility, Robustness, Equitability, Privacy, Explainability, Accountability, Transparency, and Security ((R)REPEATS) principles framework is a good reference you can use to get for the full picture on the broader ethical principles.⁶

The following sections will cover the four main MLOps principles in detail.

Traceability

Traceability means that for any given ML model run or deployment in any environment, you can look up and unambiguously find all of the following:

- Corresponding code and commits on Git
- The infrastructure used for training and serving
- ML model artifacts
- What data was used to train the ML model

In practice, this often breaks down. For example, a model may underperform months after deployment, and the team may not be able to determine what code, dataset snapshot, hyperparameters, and configurations were used. If the investigation depends on searching Slack messages, notebooks, or institutional memory, traceability has failed.

Effective traceability requires every run to automatically capture and link its inputs and outputs through a shared run identifier. [Table 1-2](#) lists the traditional ML traceability components and gives examples for each of them.

Table 1-2. Traditional ML traceability components and examples

Traditional ML traceability components	Examples
Code and commits	Git SHA
Infrastructure	Training compute and serving environment configuration as code
Data lineage	Dataset snapshots and table version
Features	Feature definitions, transformations as code, and the feature store version
Model configuration	Hyperparameters and model architecture
Artifacts	Trained models and evaluation reports
Experiment tracking	Metrics, parameters, and runs

Modern LLM and agentic systems add more layers of complexity because behavior depends not only on models and data but also on prompts, retrieval pipelines, tools, and external APIs. [Table 1-3](#) Lists LLM-based system traceability components and examples of them.

Table 1-3. LLM-based system traceability components and examples

LLM-based system traceability components	Examples
Foundation model usage	Endpoint, model version, token usage, and latency
Prompt lineage	Prompt templates, system prompts, and query enrichment
Retrieval pipelines	Retrieved chunks, ranking strategy, and metadata filters
Embedding generation	Embedding models, chunking strategy, and parsing logic
Vector database lineage	Chunk metadata, source documents, and indexing strategy
Tool and API usage	External tools invoked, parameters, and responses
Response lineage	Which model, prompt, context, and tools produced a response

Effective traceability requires that every run automatically capture and link its inputs and outputs—code commits, infrastructure, datasets, prompts, model parameters, retrieved context, tool invocations, artifacts, and evaluation results—and tie them all to a single identifier. The key is that traceability must be automatic and built into the workflow, not dependent on manual documentation.

Teams that invest in this capability spend far less time putting out fires and far more time improving their systems.

Reproducibility

Reproducibility extends traceability one step further. Traceability asks, “Can we determine what happened?” Reproducibility asks, “Can we make it happen again?”

For traditional ML systems, reproducibility means that the same code, data, configuration, and environment produce the same model and re-

sults. This makes experiments verifiable, debugging easier, and rollbacks reliable.

In practice, reproducibility takes effort. Without data snapshots, pinned dependencies, and versioned configuration, reproducing a previous result becomes difficult or impossible. Modern tooling helps address these issues. Data versioning systems, containerized environments, experiment tracking, and infrastructure-as-code (IaC) allow teams to re-create previous training conditions reliably.

For LLM and agentic systems, reproducibility becomes more complex because system behavior depends not only on code and data but also on foundation model versions, prompts, retrieval pipelines, tool usage, and inference settings such as temperature or token limits. Exact bit-for-bit reproducibility is impossible due to nondeterministic model behavior and provider-side updates. Instead, teams aim for *functional reproducibility*: the ability to re-create a system configuration that behaves consistently within an acceptable range.

It is worth noting that the boundary between traditional ML and LLM-based systems is not as clear as it might seem. Nondeterminism in deep learning predates LLMs, and evaluation for natural language processing (NLP) applications (such as translation, and summarization) was never straightforward.⁷ These were known problems, but they affected a smaller community. LLMs have made nondeterminism and evaluation difficulties impossible to ignore, simply because the scale and visibility of these systems has brought long-standing issues to the surface.

Reliability

Reliability is the principle that an ML system should function correctly and consistently throughout its lifecycle, even as code, data, and infrastructure change around it. A reliable system is one in which the team can ship changes with confidence that things won't break silently.

While reliability engineering traditionally focuses on measurement and operational resilience, reliability begins with what you can control most directly: the quality of the code itself. If the code is poor, no amount of monitoring, deployment strategizing, or incident response will be able to make up for it. You can operate around bad code for a while, but the oper-

ational burden will grow with every release, and eventually, the system will become too fragile to change safely.

Clean code ensures that models can be updated, debugged, and improved over time without introducing unintended consequences. This is not just a best practice; it is a pattern backed by decades of evidence. The industry average is about 1 to 25 errors per 1,000 lines of delivered code.

Microsoft's Applications Division achieved 0.5 defects per 1,000 lines in released products through code reading and independent testing. The space shuttle's software achieved zero defects in 500,000 lines of code.⁸ The systems with the highest stakes consistently have the cleanest code. That is not a coincidence.

Google's site reliability engineering (SRE) handbook puts it differently but arrives at the same place: simplicity is a prerequisite to reliability, and every new line of code is a liability.⁹ The tools and frameworks that ML systems depend on, however, are not simple. ML pipelines and infrastructure configurations carry inherent complexity that go beyond clean code. Code quality in this context is not about simplicity in the absolute sense but about managing unavoidable complexity through the following:

- Clear structure and modular design
- Testing at multiple levels, from unit tests to integration tests
- Consistent standards that are enforced across the team

The complexity you cannot remove is exactly the complexity that demands the most discipline.

A common counterargument is that AI-assisted code generation changes this equation—if code is cheap to produce, it can be treated as disposable. The data tells a different story:

- CodeRabbit's analysis of 470 open-source pull requests (PRs) found that AI-coauthored PRs contain approximately 1.7 times more issues overall than human-only PRs.¹⁰
- The 2024 DevOps Research and Assessment (DORA) report found that a 25% increase in AI adoption produced a 7.2% drop in delivery stability.¹¹

Generating code faster does not eliminate the cost of poor code; it shifts that cost downstream into debugging, maintenance, and production incidents. The ease of generation makes discipline more important, not less.

But code quality alone is not enough. It has to be protected by layers that reinforce each other, such as the following:

- Continuous integration and continuous deployment (CI/CD) pipelines that enforce standards automatically and consistently
- Safe deployment strategies, like canary releases and rollbacks, that limit the blast radius when something does go wrong
- Dependency management and environment pinning that prevent the kind of silent drift where a system works on one machine but fails on another

Weakness in any one layer undermines the rest. The concrete practices that operationalize this, from linting and CI enforcement to code reviews and AI-assisted tooling, are covered in the team practices section later in this chapter.

Reliability also extends into infrastructure and operational resilience, which are areas that are well established in SRE. The book *Reliable Machine Learning* by Cathy Chen et al. (O'Reilly) applies SRE principles specifically to ML systems by covering topics such as deploying across multiple availability zones, implementing authentication patterns for model endpoints, designing fallback strategies when a primary model is unavailable, and ensuring the system can scale to meet demand. These are important considerations for any production ML system, and we will touch on some of them throughout this book, but they are not its core focus.

Observability

Contrary to popular belief, the lifecycle of an AI system does not end at deployment; in many ways, deployment is where the real lifecycle begins. Once it's exposed to real users, changing data, evolving requirements, and production constraints, the system enters its most operationally complex phase. And that's where teams need observability the most.

An *observable* system is one whose internal state you can understand from its outputs, even for problems you did not anticipate. In practice, this means the system's features must include rich logging, distributed tracing, and the ability for you to ask it new questions without deploying new instrumentation. There is natural overlap between observability and the traceability principle we discussed earlier: traceability focuses on the ability to reconstruct what happened and why a decision was made, while observability is about having the instrumentation in place to detect and diagnose issues in real time. The two reinforce each other.

A core component of observability is *monitoring*: tracking known metrics and alerting when they cross a threshold. While observability helps you figure out why something went wrong, monitoring tells you that it did. It is the natural starting point, so here's a list of what an ML system needs to track:

Application-level data

Response times, error rates, throughput, and latency distribution, all of which are relevant for a model served behind an API)

Infrastructure health

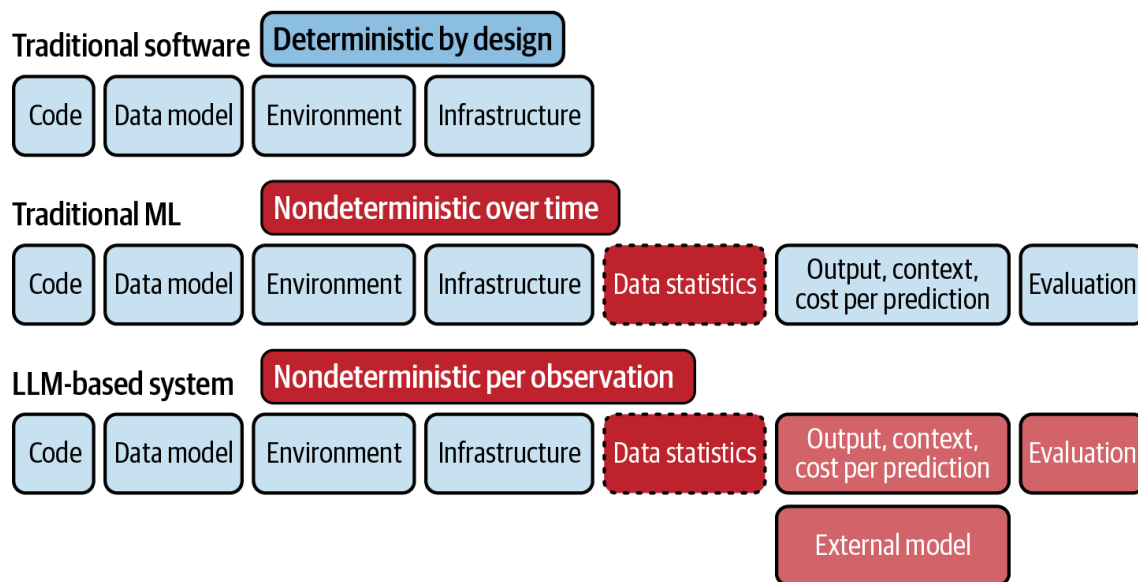
CPU health, memory utilization, and health metrics for various system components

System-relevant business metrics

For example, KPIs such as the click-through rate and conversion rate for a recommender system

ML systems differ from traditional software systems in one critical respect when it comes to monitoring: you need to monitor their data quality and detect data and model performance drift.

A software system will perform as expected as long as the related infrastructure, environment, code, and data models do not change. You have control over all that with software systems, but not with ML and AI systems. That's the key issue in ML: even if everything under your control remains unchanged, the model can start performing poorly when the data distribution shifts. [Figure 1-2](#) illustrates this difference and this challenge.



Key insight: A software system with identical properties produces identical output. An ML system cannot guarantee this. An LLM system adds further uncontrollable variables: an external model you do not own, plus nondeterministic output, retrieved context, cost, and even evaluation.

Figure 1-2. The complexity ladder: traditional software versus traditional ML versus an LLM-based system

In addition, data quality issues are some of the most common reasons why ML applications fail in production. Missing values, schema changes, incorrect labels, upstream pipeline failures, and delayed data availability can all break ML system performance, even when the underlying model and code remain unchanged. Unlike traditional software bugs, these issues often originate outside the application itself, so they're harder to detect and debug unless proper monitoring is in place.

In traditional ML, production monitoring of model performance is well established. You track metrics such as accuracy, precision, and the area under the curve (AUC) against incoming data and watch for degradation over time. When performance drops, you investigate whether the data has shifted, whether labels have changed, or whether something upstream has broken. The metrics themselves are deterministic and well defined.

For LLM-based systems, the challenge is not just tracking metrics but also defining them. Output quality is harder to quantify, and evaluation itself becomes a two-tier problem: using deterministic metrics where they apply and LLM-based evaluation, where another model judges the output, for everything else. On top of that, cost and latency vary by request and become first-class monitoring concerns in their own right. We will cover production monitoring and evaluation in depth in later chapters.

Team Practices

Principles don't survive without the right culture and knowledge sharing. Reliability, traceability, reproducibility, and observability all depend on tooling, but tools only work when teams use them consistently and intentionally. This section covers the practices that make that possible.

Documentation and Knowledge Preservation

People affected by a system should be able to understand what the system is intended to do, how it was designed, and where its limitations lie. In ML and AI systems, this is fundamentally about making systems legible to humans: the engineers who maintain them, the business stakeholders who rely on them, the auditors who review them, and the users who interact with them.

Proper documentation should include the following:

- The assumptions behind the system
- The reasoning behind architectural choices
- The intended scope of use
- The known limitations or risks

Several artifacts support this in practice:

Model cards

These explain model behavior and failure modes.

Data dictionaries

These clarify what data represents and where it originates.

Architecture decision records

These capture why specific trade-offs were made.

Documentation should live alongside the code, evolve through the same review process, and remain tightly connected to implementation.

Lightweight formats such as Markdown are effective because they are version controllable, easy to maintain, and easily consumed by both humans and AI tooling. Tools such as [MkDocs](#) can generate documentation

sites from Markdown, and documentation updates should be part of the code review process. Tools like [Swagger](#) are also useful for generating live API documentation that can be embedded into internal portals such as GitHub Pages and Confluence.

The rise of AI-powered coding assistants like [Claude Code](#) further increases the value of transparent systems. Coding assistants can summarize implementations or generate code changes, but they depend heavily on high-quality contextual information. As documentation becomes part of the development interface for both humans and AI systems, it shifts from being viewed as overhead to becoming an asset that improves development speed, onboarding, and long-term maintainability.

Ultimately, documentation is part of the culture. Organizations need norms and processes that encourage explicit communication, shared understanding, and knowledge preservation. Assigning ownership of critical documentation and conducting periodic reviews helps organizations ensure that important context does not disappear when team members leave or projects change direction.

Code Hygiene

Code quality is the foundation that all four MLOps principles rest on. A system can have perfect observability dashboards and comprehensive traceability, but if the underlying code is messy, brittle, or inconsistent, it will undermine every other investment.

Today, AI is real leverage when writing code, but leverage without discipline just breaks things faster. Protecting the main branch and enforcing automated checks before pull requests are merged creates a reliable safety net. Therefore, I advocate the following code hygiene protocols:

Pre-commit checks

You should run these as part of a CI pipeline. This point is worth clarifying. Traditionally, pre-commit hooks run locally, before a developer commits code, to enforce formatting and basic coding standards. This is ideal in theory, but in practice, developers can forget to configure pre-commit hooks or bypass them, and that can allow noncompliant code to enter the repository. Running pre-commit

checks as part of the CI pipeline ensures that coding standards are enforced consistently across the team, even when individual developers' environments are misconfigured or incomplete.

Testing

This is another essential component of automated quality assurance. Unit tests verify that individual components and functions are behaving as expected, while integration tests ensure that different parts of the system work correctly together. Together, these tests reduce the risk of failures when you're updating models, data pipelines, or serving infrastructure. We'll discuss testing in [Chapter 5](#).

Code reviews

Human-run code reviews are an important safeguard, but in practice, teams are often overloaded with PRs, and that makes thorough human review harder to sustain. LLM-based code review tools help close this gap by automatically surfacing issues, suggesting improvements, and flagging potential risks. Rather than replacing human reviewers, they act as a first-pass filter that reduces the review burden on humans and allows engineers to focus on higher-level concerns like design, correctness, and readability. As their adoption grows, these tools will become a standard complement to automated checks and human judgment.

Documentation and code quality come down to people and processes. However, MLOps principles depend heavily on tools. But MLOps isn't about which tools you pick; it's about how you use them. So next, we'll look at MLOps components, walk through some tool examples, and see how all of this maps onto the Databricks platform.

MLOps Components

You may be familiar with the Machine Learning, Analytics, and Data (MAD) landscape, which is an initiative by Matt Turck that visualizes the evolving data and AI tooling ecosystem. The first version, released in 2012, featured just 139 companies. By 2024, the landscape had expanded to more than 2,000 companies, reflecting the rapid growth of the space.

In 2025, however, the industry saw a significant consolidation, with the number of companies dropping to around 1,150. [Figure 1-3](#) highlights a subset of tools that focus on ML and AI, as the full landscape is too dense to be meaningfully read in a book format, and even this subset contains a large number of logos.

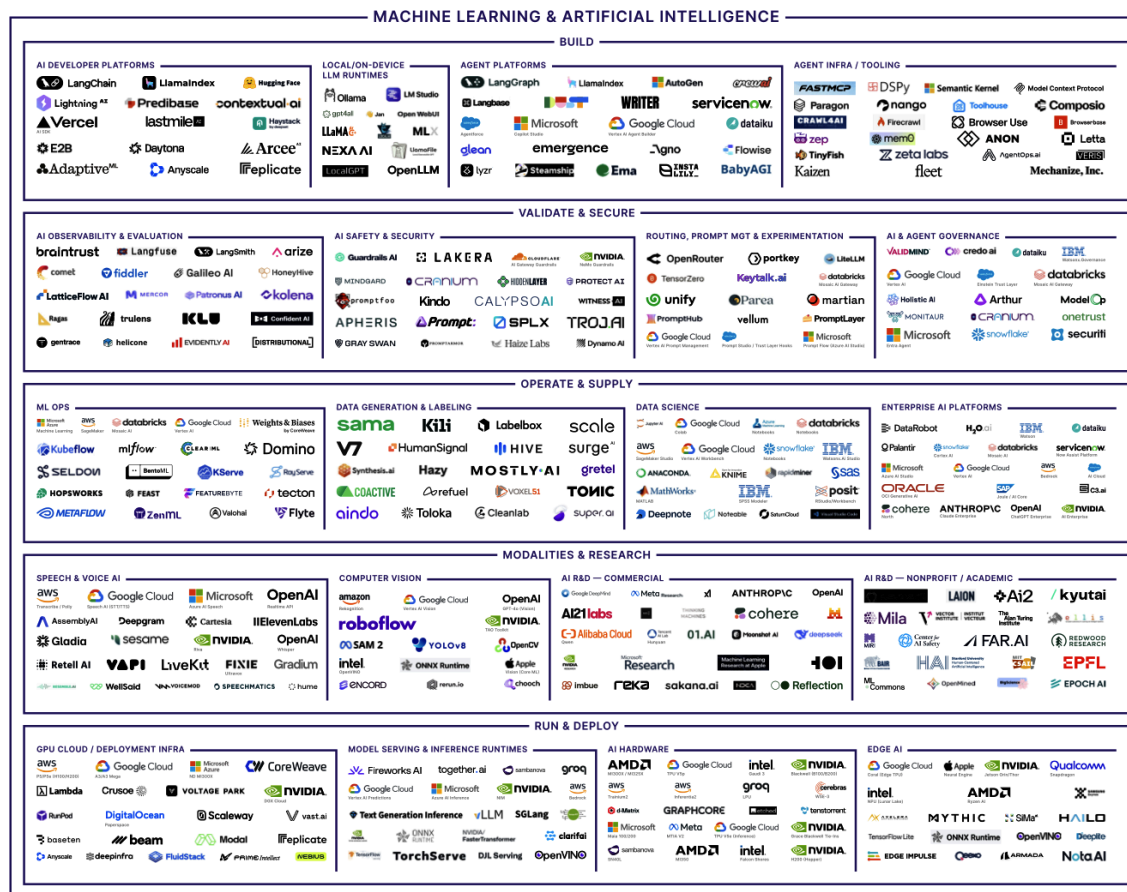


Figure 1-3. A subset of the [MAD landscape](#) focusing on ML and AI, as of 2025

When you see this many tools, it's hard to know where to start. You don't need all of them. What you need are certain types of tools, and the specific choice within each type depends on the complexity of your system. Many of these categories already exist in your DevOps stack; MLOps extends them rather than replacing them.

Each component exists to support one or more of the main MLOps principles: transparency, reliability, traceability and reproducibility, and observability. We will get to those principles after the tooling overview.

First, I'll walk through the core component categories (which are summarized in [Figure 1-4](#)) and then cover what changes when your application is built on LLMs.

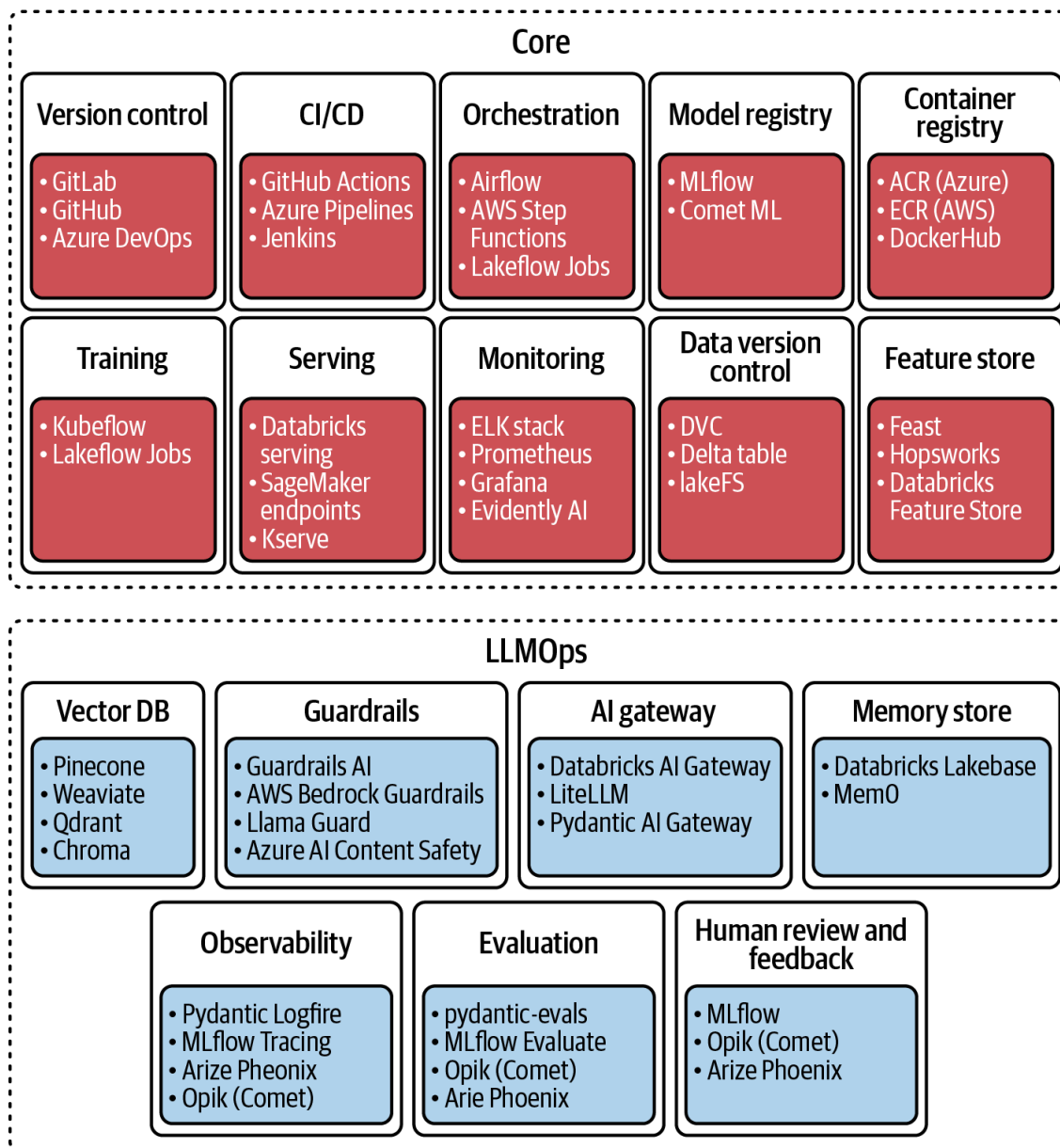


Figure 1-4. Core MLOps and LLMOps component categories spanning the ML/AI lifecycle, from code version control to monitoring

Core Components

The core components shown in [Figure 1-4](#) are universal. Whether you're deploying a classical ML model or an LLM-based application, you'll need some form of each core component. Most of them will look familiar if you've worked in software engineering; the difference is in how they're configured and what flows through them. In this section, I'll walk you through the core components

Code version control

Code version control is the practice of managing and tracking changes to source code and configuration files over time by using a version control system such as Git. It ensures the traceability and reproducibility of an ML model deployment or run. Without it, you'll have a hard time deter-

mining which exact version of the code was responsible for a particular training run, model artifact, or production error.

Platforms such as GitHub, Azure DevOps, and GitLab provide hosted environments built around Git that enable teams to collaborate on code more effectively. If you work in a corporate organization, you likely have such a platform (or even more than one) in place already. Beyond storing and managing repositories, these platforms offer features such as PRs and branch protection rules, which help enforce code quality by requiring reviews before changes can be merged into branches that allow deployment to production.

Continuous integration and continuous deployment (CI/CD)

CI/CD pipelines are automated workflows that build, test, and deploy code to production efficiently and with minimal manual intervention. In MLOps, they serve the same role as in DevOps, but what flows through them extends beyond application code to include training pipelines and serving components.

CI pipelines automatically build and test code changes whenever a PR is created, to validate that the application is still functioning as expected. No code should reach production without passing through this gate. *CD pipelines* then handle deployment, to ensure that production releases happen through a controlled, repeatable process, rather than through manual steps that are prone to error.

Most version control platforms provide built-in CI/CD solutions such as GitHub Actions, GitLab CI/CD, and Azure Pipelines. In this book, we use GitHub Actions. We'll discuss this in detail in [Chapter 5](#).

Workflow orchestration

Typical ML projects require *workflow orchestration* to manage the complex dependencies among different tasks, such as data preprocessing, feature engineering, and ML model training (which can also be parallel tasks when the model is trained separately by category). A workflow is often created for inference because inference requires a different frequency than training. This can be either *batch inference*, in which predictions get

generated and written to a database or sent over SFTP, or *real-time inference*, which means we need a workflow for endpoint deployment.

Workflow orchestrators support various triggers, such as the CRON trigger, the file arrival trigger, and the conditional trigger (which runs a job if another workflow is finished successfully).

You can use tools like Airflow, Lakeflow Jobs, Azure Data Factory, and AWS Step Functions for workflow orchestration. If your organization does not provide a workflow orchestration tool as a service, it can be a lot of work to support such a tool on your own (mainly because of security patching). Therefore, it may be wise to choose managed services, and there are always some options available if your organization uses a cloud provider.

Model and container registry

Another important aspect of MLOps is the *model registry*. This includes storing and versioning trained ML model artifacts together with additional model metadata (hyperparameters, performance metrics, etc.). You should also store technical metadata, including the code version (git commit hash) you used to train the model and references to the data you used for model training. This is essential for traceability: you will always be able to pinpoint what went wrong and when and then roll it back when you need to.

In addition to storing ML models, you may need to store Docker images as part of your MLOps setup. For example, you could use Kubernetes to run model training jobs or to serve models, you may need to provide custom Docker images to Amazon SageMaker/Azure Machine Learning (Azure ML), or you could use Docker images to run unit/integration tests in a CI/CD system. In all these scenarios, you'll need a tool to store your images, such as Azure Container Registry, JFrog Artifactory, or Amazon Elastic Container Registry.

Model training

Model training requires dedicated compute. The options depend on your setup: cloud-native services such as Azure ML, Amazon SageMaker AI

Kubeflow for Kubernetes-native ML pipelines; or Lakeflow Jobs on Databricks.

Services like Lakeflow Jobs on Databricks combine orchestration and compute provisioning in one place, which allows you to define task dependencies and schedule training runs without managing the underlying infrastructure separately. In this book, we use Databricks compute for model training, but the principles apply regardless of the platform you use.

Model Serving

Serving and training do not always need the same infrastructure.

Training is a batch workload that runs periodically, while *serving* handles real-time requests and needs to be always available. The requirements for compute, scaling, and networking are fundamentally different in these two processes.

For serving, Azure ML, SageMaker, and Databricks all provide managed solutions that simplify model deployment significantly. That said, many organizations, especially those with high Kubernetes adoption or specific latency requirements, prefer to deploy models on Kubernetes using frameworks like [KServe](#). This can be practical when other services such as the frontend already run on the same cluster and you want to minimize network overhead between them.

Monitoring

Monitoring is an essential (but sadly often ignored) piece of an MLOps setup. Monitoring in ML systems goes beyond the standard processes for monitoring regular software applications. The distinction lies in the fact that model predictions can be systematically wrong at scale, even when all typical health metrics appear to be in good condition.

Model performance should be checked regularly to prevent unexpected predictions from being consumed by end users. Tools like Azure ML, SageMaker, and Databricks Data Quality Monitoring provide features to monitor data and model drift. However, it's ideal to have all metrics in one place and use the monitoring systems already adopted by other

teams. The [ELK stack](#) and a [Prometheus](#) + [Grafana](#) setup are options that you may already have.

Data version control

Your company's compliance policies may require training data to be reproducible, and that's hard to achieve without data versioning. Simply re-running preprocessing scripts with a certain filter may not work and may erroneously include data that arrived later and was not included in the original training set, unless the scripts are designed in a specific way (which may not be straightforward).

On the other hand, data versioning tools allow you to re-create a training set exactly as it was at a given time. Tools like DVC, lakeFS, and the time travel functionality in Delta Lake help you tackle this problem.

Feature store

In general, a *feature store* is a data layer that connects data preprocessing (feature) pipelines, model training pipelines, and inference, to ensure training-inference alignment. (On the other hand, *Databricks Feature Store* is a specific product that serves as a centralized repository for ML model features; see the “Feature Engineering” section later in this chapter for details.) A feature store is useful for governance and lineage purposes (to determine what features were used for what models), and it allows for feature reusability across models.

However, not every application needs a feature store. For batch applications or real-time Model Serving that relies on precomputed features, you can achieve governance and lineage with data version control tools alone. On the other hand, having a feature store is essential in real-time systems that require on-demand feature computation or lookup with millisecond-level latency and in scenarios where you need point-in-time correctness to ensure that training data reflects exactly what was known at the time of each prediction, without leaking future information. Fraud detection is a typical example that requires both point-in-time correctness and feature computation or lookup with ms-level latency.

LLM Application Components

The core components we covered in the previous section are universal, but when you're building LLM-based applications, several additional components come into play. Some of these, like vector databases, are not exclusive to LLMs but have gained significant traction because of them. We'll cover these additional components in this section.

Vector databases

Vector databases store and retrieve embeddings, numerical representations of text, images, and other unstructured data. In LLM applications, they are most commonly used for retrieval-augmented generation (RAG), in which relevant documents are retrieved and injected into the prompt to ground the model's response in internal knowledge. They are not limited to LLMs, though; recommender systems have relied on vector similarity search for years.

Tools like Pinecone, Weaviate, Qdrant, and Databricks AI Search (formerly Databricks Vector Search) make different tradeoffs when it comes to deployment, integration, filtering capabilities, and scalability.¹² Some traditional databases like PostgreSQL (with [pgvector](#)) also support vector search, which can be practical when you want to avoid introducing a separate system. On Databricks, Lakebase offers managed PostgreSQL with pgvector support, which can be a more cost-efficient alternative to Databricks AI Search for workloads that don't need a dedicated always-on endpoint.

Guardrails

Guardrails validate model inputs and outputs at inference time to enforce safety, compliance, and quality standards. They can prevent harmful or off-topic responses, enforce structured output formats, check for personally identifiable information (PII), and ensure that the model's behavior stays within business rules and domain-specific constraints.

Conceptually, guardrails are closely related to evaluation: they are evaluation checks that run in the production path, rather than offline. The reason they exist as a separate component is that certain controls, like PII detection and safety filtering, need to be enforced consistently at the plat-

form level rather than left to individual developers to implement separately in each application.

The tooling splits into two categories:

- Managed platforms like Amazon Bedrock Guardrails, Azure AI Content Safety, and Databricks AI Guardrails intercept requests and block non-compliant inputs or outputs before they reach the user. The tools in this category enforce controls by default, regardless of what the application code does.
- Tools like [Guardrails AI](#), OpenAI's Moderation API, and models like Llama Guard provide building blocks for implementing similar checks as part of the serving layer.¹³ for details. In this category, enforcement becomes the responsibility of the application or platform team. It must either integrate checks into the application code directly or build a shared serving layer that enforces them centrally.

AI gateways

An *AI gateway* sits between your application and one or more LLM providers. It handles routing, fallback logic, rate limiting, API key management, and cost tracking among models and endpoints. This becomes important as soon as you start using more than one model or provider or when you need visibility into token consumption and spend.

Databricks Unity AI Gateway integrates natively with the Databricks platform. LiteLLM is an open-source, self-hosted alternative. [Pydantic AI Gateway](#) provides cost control and routing with an open-source core (AGPL-3.0). It can be managed through Pydantic Logfire or self-hosted for enterprise.

Memory stores

LLMs are stateless by default. Every API call is a blank slate with no knowledge of previous interactions. A *memory store* solves this problem by persisting conversation history, user context, and accumulated knowledge across sessions, so the application can behave as if it remembers previous sessions.

Databricks Lakebase provides a managed PostgreSQL backend for storing agent state, conversation history, and vector-based context. Mem0 takes a different approach with a three-tier memory model (user, session, and agent scopes) backed by a hybrid of vector, graph, and key-value stores. Which one you should use depends on whether you need infrastructure-level storage or a higher-level memory extraction and recall system.

Observability tools

LLM *observability* goes beyond traditional application monitoring. You need to trace full execution paths through your application: API calls to LLM providers, tool invocations, document retrieval steps, and chain-of-thought reasoning. Without this knowledge, debugging why an agent gave a wrong answer or took an unexpected action becomes guesswork. And while we've listed them under LLM application components, LLM observability tools can be equally useful for traditional ML serving with complex inference logic.

Major tools like Pydantic Logfire, MLflow Tracing, Opik (from Comet), and Arize Phoenix are all built on OpenTelemetry, which means any of them can trace your full application stack, not just the AI layer. And their core tracing capabilities are converging. The main differentiator is ecosystem integration: MLflow Tracing connects to Databricks Unity Catalog for governance, lineage, and access control, while Logfire ties into the Pydantic stack. MLflow, Opik, and Arize Phoenix are open-source and can be self-hosted. Pydantic Logfire is available as a managed service or can be self-hosted with an enterprise license.

Evaluation tools

Evaluating LLM outputs is fundamentally harder than evaluating traditional ML models. There is no single accuracy metric. Outputs are open ended, context dependent, and often subjective. Evaluation in LLM Ops combines automated metrics (LLM based or code based) with human scoring to build confidence that the system is behaving as intended.

Tools like Pydantic Evals, MLflow Evaluate, and Opik (by Comet) all support LLM-as-judge and custom metrics. Pydantic Evals integrates with the Pydantic ecosystem and Logfire. MLflow Evaluate ties into the Databricks platform. Opik is open source and can be self-hosted.

Human review and feedback

Automated evaluation provides scale, but human judgment remains the ground truth. Reviewers inspect model outputs, approve or reject agent actions, and provide corrections that feed back into improving the system. This is especially relevant for agentic applications in which the model takes actions (making API calls, performing database writes, and sending messages) that are difficult or impossible to reverse.

Most tools we've mentioned in the observability and evaluation sections also support human review as a core capability. MLflow, Opik, and Arize Phoenix all provide trace annotation and human scoring workflows. The boundaries between observability, evaluation, and human review are blurring: in practice, a reviewer looks at a trace, and scores it, and that score becomes both an evaluation data point and a feedback signal. The tooling reflects this convergence.

Choosing Your Tools

We've covered a lot of ground in this chapter: principles, team practices, and tools across both the core and LLM-specific components. The landscape is broad, and it's easy to feel overwhelmed. So how do you decide which tools to adopt?

Start with what your organization already has. The goal isn't to chase every new tool but rather to avoid unnecessary complexity in an already crowded space. Only introduce something new when there's a clear gap that your current stack can't fill. This matters more than it sounds. Every tool comes with onboarding, maintenance, integration work, and eventually migration. The ecosystem moves fast. What used to shift every few years now shifts every few months, driven by the rapid adoption of AI. The more tools you take on, the harder it becomes to adapt when things change again, and they will. A lean, intentional stack is easier to evolve than one built on every promising option that comes along.

This is also where platforms start to make a strong case for themselves. Rather than stitching together separate tools for each component, a platform like Databricks consolidates many of them under one roof. That doesn't mean it replaces everything, but it reduces the number of moving parts you need to manage independently. In the next section, we'll walk

through how the Databricks platform maps to the components we’ve discussed and what the transition from general tooling to a platform-native approach looks like.

Databricks MLOps Components

Databricks provides all the features you need for MLOps. [Table 1-4](#) maps the MLOps and LLMOps landscape component categories from [Figure 1-4](#) (earlier in the chapter) to their Databricks equivalents.

Table 1-4. Mapping MLOps and LLMOps component categories to Databricks

MLOps and LLMOps component categories	Databricks equivalents
Workflow orchestration	Lakeflow Jobs
Model registry	MLflow and Unity Catalog
Model training	Lakeflow Jobs (using Databricks compute)
Serving	Model Serving, Feature Serving, Foundation Models
Monitoring	Data Quality Monitoring
Data version control	Delta tables
Feature stores	Feature Engineering in Unity Catalog
Vector DB	Databricks AI Search
Guardrails	Databricks AI Guardrails
AI gateways	Databricks Unity AI Gateway
Memory stores	Lakebase
Observability tools	MLflow Tracing
Evaluation tools	MLflow Evaluate
Human review and feedback	MLflow

In the next chapters, we will go through all of these components in detail and show you how to combine them in a way that promotes MLOps prin-

ciples. In the remainder of this section, we'll cover the components from [Table 1-4](#) in more detail and then discuss cross-cutting components that don't map to a single category.

Lakeflow Jobs

Lakeflow Jobs is an orchestration system that's integrated into the Databricks platform. There are three main concepts related to Lakeflow Jobs that you need to know about:

Jobs

A job is a primary unit of orchestration that supports various triggers.

Triggers

These include the CRON trigger, the file arrival trigger, table updates, the job dependency trigger (which runs the job when another job is completed), and the API trigger (which starts a job on API call).

Tasks

Databricks jobs consist of tasks that support various complex scenarios. Each task can perform a variety of operations. Here are some that are commonly used for ML workflows:

- A Databricks notebook with Python code and/or SQL queries
- A Python script or a wheel
- Lakeflow Spark Declarative Pipelines (formerly Delta Live Tables) for Extract, Load, Transform (ELT) pipelines
- If/else conditions that can be handy if, for example, the workflow must run only on certain days or certain hours and the logic goes beyond what CRON definition supports
- For-each loops that can be useful when you need to loop over the same task with different parameters and these sub-tasks must be executed in parallel

Databricks is constantly improving the Lakeflow Jobs functionality, and new features, including new types of tasks, are released frequently.

Databricks Compute

You can specify the compute that will be used to run each of the tasks of a Lakeflow job. For some tasks (for example, a Python task), you can choose either a serverless compute or a classic compute.

Classic compute versus serverless compute

Classic compute is configured by the users, and it runs within the customer's cloud environment. The costs for classic compute are distributed between Databricks and the cloud provider.

For a classic compute, users have all the freedom to choose the configuration of the following:

- The *runtime* (the environment used to execute the code) that defines the Python version, Spark version, and OS version
- Pre-installed libraries
- The worker and driver type
- The autoscaling policy
- Advanced settings (for example, init scripts or Docker images)

Serverless compute means that the compute infrastructure is fully managed and optimized by Databricks. In this setup, there are no underlying infrastructure costs billed through the customer's cloud provider.

Serverless compute also often delivers a better price/performance ratio than classic, customer-managed clusters. In addition, serverless workloads start almost instantly, so users don't have to wait for clusters to spin up. While users have limited control over low-level compute settings and runtime configuration, they can still manage Python versions and libraries through the environment settings provided with serverless compute.

Serverless compute works well for development, data engineering, and AI engineering workloads. However, it is not always suitable for certain ML model training tasks. Although Databricks is actively investing in its serverless offering—and the list of limitations is shrinking rapidly—some constraints remain.

One of the main current limitations on serverless compute is the maximum available size of the driver node. Many Python-based ML training workflows rely on libraries such as scikit-learn and the Light Gradient-Boosting Machine (LightGBM), which typically operate on in-memory data structures such as Pandas or Polars DataFrames. These libraries are not distributed, and they execute computations on the driver node. As a result, the maximum available driver size in a serverless environment may not always be sufficient for memory-intensive training workloads.

Traditional ML training workflows often still require classic compute for greater flexibility and resource control. In contrast, many modern AI workloads can run efficiently on serverless infrastructure. Since we focus on both traditional ML and AI applications in this book, we'll use both types of compute and discuss implications for development.

All-purpose versus job compute

For a task in a Lakeflow job that utilizes standard compute, you can choose between all-purpose (often named as interactive) compute and job compute. Overall, you should use a job compute for Lakeflow jobs and all-purpose clusters for development. While the costs associated with your cloud provider will be the same, the Databricks costs, measured in Databricks Units (DBUs), will be significantly lower.

MLflow

MLflow is one of the oldest and most popular tools for experiment tracking and model registry. MLflow is an open-source tool that was originally created by Databricks, and it is also available as part of the Databricks data and AI platform.

Experiment tracking is managed at the Databricks workspace level. It allows you to store parameters, metrics, and metadata that belong to an experiment run. A model must be logged using supported flavors in the experiment tracking system before it can be registered.

While the model registry was previously also at the workspace level (which is now deprecated), the recommended approach today is to register models in the Unity Catalog using the MLflow library.

Given that organizations increasingly build AI applications powered by large language models, the *MLflow Tracing* product has been introduced as a key component for ensuring traceability and reproducibility. It integrates with many GenAI libraries and helps developers understand and debug what happens below the abstraction layer. We'll talk about MLflow Tracing in detail in [Chapter 8](#).

Feature Engineering

Databricks Feature Store is a centralized repository for storing and sharing ML model features. Workspace Feature Store is marked as legacy and has been deprecated, and Databricks recommends Feature Engineering in Unity Catalog for handling model features.

Databricks provides a Python API to interact with the Databricks Feature Engineering in Unity Catalog. It is a key component that you can use to do the following:

- Create, update, and delete feature tables and feature specifications
- Create a training dataset
- Log an MLflow model packaged with information on how a training set was built and then use it for batch scoring

Feature engineering functionality is especially useful when out-of-the-box automatic feature lookup is required for real-time model inference.

Databricks Model Serving

Databricks provides Model and Feature Serving functionality, which can significantly simplify the deployment.

Databricks Model Serving is designed for real-time model inference. It requires you to register a model in Unity Catalog using MLflow, and after that, you can deploy the model behind a serving endpoint using MLflow's serving capabilities.

You can also use Databricks Model Serving to access proprietary foundation models from other providers such as Azure OpenAI, Amazon Bedrock, and Anthropic, as well as to serve (fine-tuned) open source foundation models and agents. Databricks-hosted pay-to-go foundation models

are also available through the platform. We'll talk about these options in detail in [Chapter 7](#).

Databricks Model Serving for ML applications integrates with feature engineering functionality to enable automatic feature lookup from tables stored in the [Databricks Online Feature Stores](#). These stores are built on *Lakebase*—a fully managed PostgreSQL-based online transaction processing (OLTP) database engine—and they contain read-only copies of feature tables that are optimized for low-latency access. The tables are synchronized from Delta tables continuously or based on a trigger. The lookup logic is predefined within the feature engineering pipeline, so features can be retrieved automatically during inference. Lakebase is used in different parts of the Databricks ecosystem beyond Feature Serving, and we discuss it in the section [“Cross-Cutting Components”](#).

If you need more flexibility, you can implement feature retrieval logic directly within the model definition. In such cases, users are responsible for managing authentication and access to the underlying data sources themselves. We will implement an example of this approach in [Chapter 9](#), where we will use Lakebase as memory for an AI agent.

Databricks Feature Serving is a great fit when you require a batch serving architecture. Precomputed features are stored in a feature table in Unity Catalog. A table in Databricks Online Feature Stores and a feature specification (FeatureSpec, a user-defined set of features and functions) are used behind the Feature Serving endpoint.

Data Quality Monitoring

Data Quality Monitoring helps you monitor data drift and detect model performance deterioration if the ground truth is provided. When you enable Data Quality Monitoring on a table, the profile metrics table, the drift metrics table, and a customizable dashboard are created out-of-the-box.

Data Quality Monitoring can be applied and incrementally updated on time series data or inference data (e.g., data containing information about payload for a serving endpoint). You can also use the third type of supported profile, a snapshot, for all other types of tables. The data quality

metrics will then be calculated for all data in the table, and all the data in the table will be processed with every refresh.

It is possible to set alerting on the profile or drift metrics table and get notified in case a certain drift metric exceeds a predefined threshold.

Databricks AI Search

The Databricks AI Search functionality in Databricks allows you to create an AI Search index from a Delta table. To use an AI Search index, you need to create an AI Search endpoint first. Just like a Model Serving endpoint, it uses serverless compute. You can query the index by using a representational state transfer API (REST API) or the Databricks vector search Python client.

Like other vector databases, AI Search on Databricks uses the hierarchical navigable small world (HNSW) algorithm for its approximate nearest neighbor search, and it supports hybrid keyword similarity search.

Like an online table, the AI Search index can be updated automatically every time there is an update in a delta table.^{[14](#)} Alternatively, it can be updated based on a trigger.

Unity AI Gateway

Unity AI Gateway is the central AI governance layer on Databricks for agents, LLM endpoints, model context protocol (MCP) servers, and coding agents. It provides a unified API across models from different providers—including Anthropic, OpenAI, Google, and open-source models—with consistent policy application. This eliminates the need for organizations to duplicate configurations for each provider.

Unity AI Gateway allows organizations to analyze usage, configure permissions, enforce guardrails, and manage capacity across providers. End-to-end observability is built in: you can monitor LLM and MCP calls, track usage, and attribute costs across models, teams, and workflows.

Databricks AI Guardrails

The AI Guardrails in Unity AI Gateway allow organizations to enforce safety, compliance, and business policies on model inputs and outputs. Guardrails can help prevent harmful or offensive content, detect or block sensitive information, keep conversations on topic, and ensure that responses align with organizational or branding requirements.

Databricks provides built-in guardrails for PII redaction and blocking, unsafe content filtering, jailbreak detection, and hallucination detection, along with support for custom prompt-based guardrails that are tailored to organization-specific policies. We will discuss guardrail configuration as part of the AI Gateway setup in [Chapter 9](#).

Cross-Cutting Components

Each of the components we've described so far maps to one of the specific component categories in the MLOps and LLMOps landscape that we summarized in [Table 1-4](#)). However, several Databricks components play a different role: they cut across multiple categories, or they provide infrastructure that enables other components to work together. These components don't fit neatly into a single row in the table, but they play an important role in how the platform functions as a whole. We cover these cross-cutting components in this section.

Unity Catalog

Unity Catalog is a governance layer for data and AI that is built into the Databricks platform and was open-sourced in 2024. It provides centralized access control, lineage tracking, auditing, and discoverability for data and AI assets across Databricks workspaces. These assets include ML models, functions, Delta tables, views, and files stored in Databricks Volumes.

When enabled, Unity Catalog operates transparently beneath every data interaction. It enforces permissions when tables are queried, tracks lineage as data moves through pipelines, and provides centralized governance and auditability across data and AI assets.

Rather than fitting into a single component of the MLOps stack, Unity Catalog underpins many core capabilities across the platform, including

model registry, feature engineering, data versioning, serving, and AI governance. Its introduction was one of the most significant changes in the Databricks ecosystem because it substantially simplified the implementation of production-grade MLOps practices.

Declarative Automation Bundles

Declarative Automation Bundles (DABs) are the “glue” that helps to deploy Databricks resources, such as Lakeflow jobs, with all the dependencies.

In general, there are several ways of deploying Lakeflow jobs without DABs: you could use Terraform or Databricks APIs, for example, but then, you would need to take care of a lot of additional components. For example, if you wanted to execute a Python script (let’s call it `main.py`) that requires a *project_config.yml* file, you would have to ensure that all files were uploaded to the workspace or Unity Catalog Volumes before deploying the job, and you would also have to verify that the location of those files was referenced in the job definition.

Declarative Automation Bundles help you manage these dependencies. It uploads all the files that belong to the project to a path in the workspace, which users can then define. It is a recommended and fully supported way to deploy jobs and other resources on Databricks.

We discuss Declarative Automation Bundles in detail in [Chapter 5](#).

Lakebase

Lakebase is a fully managed Postgres database that’s built into the Databricks platform. Databricks acquired the technology behind Lakebase through its acquisition of Neon,¹⁵ a serverless Postgres company, and it thereby brought a production-grade serverless database engine into the platform. Lakebase serves multiple roles across the Databricks ecosystem: it powers the Online Feature Stores with read-only copies of feature tables that are optimized for low-latency access, and it can be used as a memory store for AI agents, which is an approach we discuss in [Chapter 7](#).

Lakebase also supports several capabilities that make it well suited for operational AI workloads. It scales to zero when not in use, which means you won't pay for idle infrastructure. That's particularly relevant for development environments or workloads with variable traffic patterns. It also lets you restore databases from a previous state, so it provides a safety net for operational workloads. Because Lakebase is fully managed, there is no infrastructure to provision or maintain; compute and storage are handled by the platform.

Before the introduction of Lakebase, the Databricks platform had a gap: there was no native solution for the low-latency, transactional access patterns required for online feature serving and agent memory. Organizations had to rely on external databases for these use cases, and that added operational complexity and broke the governance boundary.

Databricks Apps

Databricks Apps provide a hosting environment for custom applications built on Databricks serverless infrastructure. As Databricks standardizes AI application and agent deployment around Apps, they are becoming an important component of production AI architectures. Databricks Apps simplify deployment by providing built-in open authorization (OAuth) support, permissions management, and governance integration out of the box.

Databricks Apps are also the recommended way to host custom Model Context Protocol (MCP) servers. While Databricks provides managed MCP servers for its own platform capabilities, organizations often need to expose internal APIs, databases, or custom business logic as tools that agents can call.

Even though the code examples in this book do not utilize Databricks Apps directly, they are becoming an increasingly important part of the Databricks AI ecosystem and are likely to play a central role in how production AI systems are deployed and integrated going forward.

Conclusion

In this chapter, we walked through the MLOps principles and components, plus the tools within the Databricks ecosystem that are useful for MLOps. One of the pitfalls I have seen teams fall into is taking a tools-first approach without having a clear understanding why they need the tool and how to use it to support the principles. I wouldn't recommend adding any tool (especially if it means extra costs and maintenance effort) unless it simplifies the development lifecycle or helps to make your system more reliable.

Next, before we proceed with practical implementation, I will walk you through the different developer tools that are available on Databricks and how you can use them to speed up the development cycle.

- ¹ D. Sculley et al. "Hidden Technical Debt in Machine Learning Systems," *Advances in Neural Information Processing Systems*, 2015.
- ² Read more about Uber's Michelangelo in [the company's engineering blog](#).
- ³ Read more about scaling ML productivity at LinkedIn in [the company's engineering blog](#).
- ⁴ Read more about Netflix's ML platform in [the company's blog](#).
- ⁵ SphereOI, *How Zillow Could Have Avoided Its \$500M AI Mistake*, September 1, 2022, <https://oreil.ly/ES7-w>.
- ⁶ For more information about the framework, consult the Linux Foundation's AI and data [blog announcement of their principles for trusted AI](#).
- ⁷ One source of nondeterminism in deep learning inference is floating-point nonassociativity: because floating-point addition is not perfectly associative, graphics processing unit (GPU) operations that reduce large arrays can produce different results depending on accumulation order, which varies with batch size and parallelization strategy. This is a solvable engineering problem. Thinking Machines Lab has demonstrated that batch-invariant kernel implementations can achieve fully deterministic LLM inference, though at a performance cost. See Horace He and Thinking Machines Lab's ["Defeating Nondeterminism in LLM Inference"](#) for more

information. For a practical walkthrough, see also Austin Zaccor’s [“On the Topic of LLMs and Non-Determinism”](#).

8 S. McConnell, *Code Complete*, 2nd ed. (Cisco Press, 2004), ch. 22, “Developer Testing.”

9 The handbook is available on [Google’s dedicated SRE site](#).

10 For more details, consult the [CodeRabbit State of AI vs. Human Code Generation Report](#).

11 For more details, read [the full DORA report](#).

12 Superlinked provides an [extensive comparison of different vector databases](#).

13 Check out the [Llama Guard 4 model card](#)

14 Continuous sync mode is not supported for [storage optimized endpoints](#).

15 For more details, check out [Databricks’ announcement](#).

Previous chapter

< [Preface](#)

Next chapter

[2. Developing on Databricks](#) >